

DOSSIER DE PROJET PROFESSIONNEL

PLATEFORME ZENBOOK

Application Web Sécurisée de Réservation de Ressources

Examen : Titre Professionnel Développeur Web et Web Mobile (DWWM)

Niveau : Équivalent BAC+2 (Niveau 5)

Document réglementaire remis aux membres du jury décrivant la conception et la réalisation technique du projet présenté en amont de la session.

Candidat : VALENTIN Mickael

Session : 2026

Centre de formation : Centre Européen De Formation

Framework utilisé : Symfony 6.4 LTS / Doctrine ORM / Twig

Table des Matières

1. Introduction Générale et Contexte	Page 3
2. Activité Type 1 : Développer la partie Front-End	Page 4
2.1. Maquetter des interfaces utilisateur web ou web mobile	Page 4
2.2. Réaliser des interfaces utilisateur statiques web ou web mobile	Page 4
2.3. Développer la partie dynamique des interfaces utilisateur	Page 5
3. Activité Type 2 : Développer la partie Back-End	Page 6
3.1. Mettre en place une base de données relationnelle	Page 6
3.2. Développer des composants d'accès aux données SQL	Page 7
3.3. Développer des composants métier côté serveur	Page 7
4. Gestion de la Sécurité & Conformité RGPD	Page 9
5. Conclusion, Bilan et Perspectives	Page 10

1. Introduction Générale et Contexte

Le présent dossier s'inscrit dans le cadre de la validation du Titre Professionnel Développeur Web et Web Mobile (DWWM). Il détaille la conception, l'architecture et l'implémentation de l'application **ZenBook**, une plateforme de réservation de ressources (locaux, équipements, matériels pédagogiques).

1.1. Problématique et Besoin

Dans de nombreuses structures, la planification des ressources partagées engendre des frictions : conflits d'horaires, manque de visibilité sur les disponibilités. ZenBook répond à ce besoin en offrant un outil moderne, automatisant le contrôle des chevauchements de créneaux et cloisonnant les accès selon le profil des utilisateurs.

1.2. Spécifications Fonctionnelles

L'application cible deux types d'acteurs distincts via un système de contrôle d'accès basé sur les rôles (RBAC) :

- **Utilisateurs connectés (ROLE_USER)** : Consultation du catalogue des ressources disponibles, visualisation des plannings, soumission d'une demande de réservation par formulaire.
- **Administrateurs (ROLE_ADMIN)** : Gestion complète (CRUD) des ressources, supervision globale de l'ensemble des réservations de la structure.

1.3. Choix de la Stack Technique

Le framework de choix est **Symfony 6.4 LTS**. Ce choix se justifie par l'intégration native de patrons de conception éprouvés (MVC, Injection de Dépendances), son composant de sécurité avancé, l'utilisation de l'ORM **Doctrine** pour l'abstraction de la base de données, et le moteur de templates **Twig**.

2. Activité Type 1 : Développer la partie Front-End

L'Activité Type 1 englobe l'ensemble des étapes menant à la production d'une interface utilisateur web moderne, ergonomique, adaptative et sécurisée.

2.1. Maquetter des interfaces utilisateur web ou web mobile

Compétence C1 : Maquetter des interfaces

La phase de conception a débuté par l'analyse de l'expérience utilisateur (UX). Des wireframes basse fidélité ont d'abord mis à plat l'enchaînement des écrans et le parcours utilisateur pour effectuer une réservation. Une maquette haute fidélité a ensuite été produite sous Figma, définissant une charte graphique épurée.

2.2. Réaliser des interfaces utilisateur statiques web ou web mobile

Compétence C2 : Réaliser des interfaces statiques

La transcription des maquettes s'est appuyée sur une structure HTML5 sémantique. L'intégration des feuilles de style s'est faite via des règles CSS3 personnalisées (Flexbox et CSS Grid). La contrainte d'adaptativité (Responsive Web Design) a été gérée par une approche Mobile-First.

2.3. Développer la partie dynamique des interfaces utilisateur

Compétence C3 : Développer le dynamisme

Le dynamisme de l'interface de ZenBook repose sur l'exploitation conjointe de **Twig** et de scripts JavaScript asynchrones. Twig apporte une structure modulaire grâce au mécanisme d'héritage de templates. Il permet d'injecter dynamiquement les données fournies par les contrôleurs tout en assurant une sécurité contre les failles XSS grâce à l'échappement automatique.

3. Activité Type 2 : Développer la partie Back-End

L'Activité Type 2 est dédiée à l'implémentation de l'architecture serveur, de la logique métier complexe et de la persistance des données.

3.1. Mettre en place une base de données relationnelle

Compétence C4 : Mettre en place une BDD

La structure de la base de données relationnelle (système MariaDB) est modélisée pour assurer une intégrité absolue. Le Modèle Physique des Données (MPD) s'articule autour de trois entités maîtresses : **User**, **Resource** et **Reservation**.

Table	Propriété / Colonne	Type SQL	Contrainte / Clé
user	id	INT AUTO_INCREMENT	Clé Primaire
	email	VARCHAR(180)	Unique
	roles	JSON	-
resource	id	INT AUTO_INCREMENT	Clé Primaire
	name	VARCHAR(255)	Non Null
reservation	id	INT AUTO_INCREMENT	Clé Primaire
	start_date	DATETIME	Non Null
	end_date	DATETIME	Non Null
	client_id	INT	Clé Étrangère
	resource_id	INT	Clé Étrangère

Le projet utilise l'outil de Migrations de Doctrine ORM pour modifier et versionner le schéma SQL à l'aide de classes PHP d'entités contenant des Attributs.

3.2. Développer des composants d'accès aux données SQL

Compétence C5 : Composants d'accès aux données

L'accès aux données s'opère via les classes Repository générées par Doctrine. Pour répondre à l'exigence métier critique d'évitement des doublons, une méthode sur mesure a été développée dans le

ReservationRepository en utilisant le QueryBuilder de Symfony, protégeant ainsi l'application contre les injections SQL via le liage de paramètres.

```
// src/Repository/ReservationRepository.php
namespace App\Repository;

use App\Entity\Reservation;
use Doctrine\Bundle\DoctrineBundle\Repository\ServiceEntityRepository;

class ReservationRepository extends ServiceEntityRepository
{
    public function findOverlappingReservations($resource, $start, $end)
    {
        return $this->createQueryBuilder('r')
            ->andWhere('r.resource = :res')
            ->andWhere('r.startDate < :end')
            ->andWhere('r.endDate > :start')
            ->setParameter('res', $resource)
            ->setParameter('start', $start)
            ->setParameter('end', $end)
            ->getQuery()
            ->getResult();
    }
}
```

3.3. Développer des composants métier côté serveur

Compétence C6 : Composants métier serveur

La logique applicative est orchestrée par le BookingController. Sa responsabilité principale est d'intercepter la requête HTTP de réservation, de vérifier la disponibilité, puis de persister la transaction ou de lever une alerte.

```
// src/Controller/BookingController.php
namespace App\Controller;

use App\Entity\Reservation;
use App\Entity\Resource;
use App\Repository\ReservationRepository;
use Doctrine\ORM\EntityManagerInterface;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Request;

class BookingController extends AbstractController
{
    public function book(Resource $resource, Request $request, ReservationRepository $repo, EntityManagerInterface $em)
    {
        $start = new \DateTime($request->request->get('start'));
        $end = new \DateTime($request->request->get('end'));

        $overlap = $repo->findOverlappingReservations($resource, $start, $end);

        if (count($overlap) > 0) {
            $this->addFlash('danger', 'Ce créneau horaire est indisponible.');
```

```
            return $this->redirectToRoute('app_resource_show', ['id' => $resource->getId()]);
        }

        $reservation = new Reservation();
        $reservation->setResource($resource);
        $reservation->setClient($this->getUser());
        $reservation->setStartDate($start);
        $reservation->setEndDate($end);

        $em->persist($reservation);
        $em->flush();

        return $this->redirectToRoute('app_home');
    }
}
```


4. Gestion de la Sécurité & Conformité RGPD

Une attention extrême a été portée aux mécanismes de défense applicative.

4.1. Le pare-feu de Symfony et le système RBAC

Le contrôle d'accès est centralisé dans le fichier de configuration de la sécurité, interdisant l'accès aux routes d'administration aux utilisateurs non authentifiés.

4.2. Hachage des secrets cryptographiques

L'application délègue le hachage des mots de passe à la configuration native de Symfony qui sélectionne dynamiquement l'algorithme le plus robuste (Argon2id ou Bcrypt).

5. Conclusion, Bilan et Perspectives

Le développement de la plateforme ZenBook a permis de valider concrètement l'ensemble des compétences des deux activités types du REAC. Les jalons futurs prévoient l'intégration de notifications par courriel ainsi que le développement d'une API RESTful.

Fin du document technique officiel — Centre Européen De Formation